Automated Assessment of Residual Plots with Computer Vision Models

Weihao Li[a,b], Dianne Cook[a], Emi Tanaka[b], Susan VanderPlas[c,d], Klaus Ackermann[a]

[a]Department of Econometrics and Business Statistics, Monash University, Clayton, VIC, Australia; [b]Research School of Finance, Actuarial Studies and Statistics, Australian National University, Acton, ACT, Australia; [c]Department of Statistics, University of Nebraska, Lincoln, Nebraska, USA; [d]Department of Mathematics & Statistics, Utah State University, Logan, Utah

ARTICLE HISTORY

Compiled July 14, 2026

ABSTRACT

Plotting the residuals is a recommended procedure to diagnose deviations from linear model assumptions, such as non-linearity, heteroskedasticity, and non-normality. The presence of structure in residual plots can be tested using the lineup protocol to do visual inference. There are a variety of conventional residual tests, but the lineup protocol, used as a statistical test, performs better for diagnostic purposes because it is less sensitive and applies more broadly to different types of departures. However, the lineup protocol relies on human judgment which limits its scalability. This work presents a solution by providing a computer vision model to automate the assessment of residual plots. It is trained to predict a distance measure that quantifies the disparity between the residual distribution of a fitted classical normal linear regression model and the reference distribution, based on Kullback-Leibler divergence. Evaluation using data from a prior human-subject experiment shows the model exhibits lower sensitivity than conventional tests but higher sensitivity than human visual tests. Several examples from classical papers and contemporary data demonstrate the method's practical utility in automating residual diagnostics.

KEYWORDS

statistical graphics; data visualization; visual inference; computer vision; machine learning; hypothesis testing; regression analysis; cognitive perception; simulation;

practical significance

1. Introduction

Plotting residuals is commonly regarded as a standard practice in linear regression diagnostics (Belsley, Kuh, and Welsch 1980; Cook and Weisberg 1982). This visual assessment plays a crucial role in identifying whether key model assumptions, such as linearity, homoscedasticity, and normality, are reasonable. It also helps in understanding the goodness of fit and various unexpected characteristics of the model.

Generating a residual plot in most statistical software is often as straightforward as executing a line of code or clicking a button. However, accurately interpreting a residual plot can be challenging, as some features are a result of predictor characteristics and natural variation, while others indicate violations of model assumptions (Li et al. 2024). For example, in Figure 1, the residual plot displays a triangular left-pointing shape. The distinct difference in the spread of the residuals across the fitted values might lead analysts to suspect heteroskedasticity. In this case, however, the fitted regression model is correctly specified, and the triangular shape is actually a result of the skewed distribution of the predictors, rather than indicating a flaw in the model.

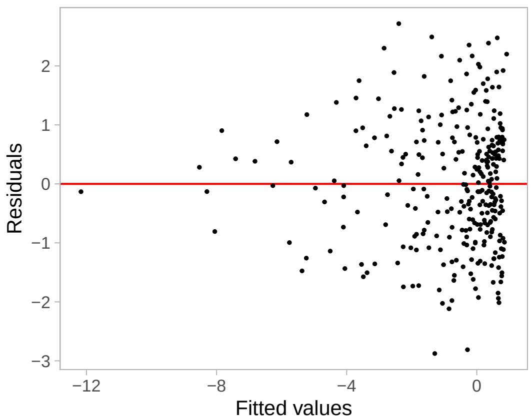

Figure 1. An example residual vs fitted values plot (red line indicates 0 corresponds to the x-intercept, i.e. $y = 0$). The vertical spread of the data points varies with the fitted values. This often indicates the existence of heteroskedasticity, however, here the result is due to skewed distribution of the predictors rather than heteroskedasticity. The Breusch-Pagan test rejects this residual plot at 5% significance level (p-value $= 0.046$).

The concept of visual inference, as proposed by Buja et al. (2009), provides an inferential framework for assessing whether residual plots contain visual patterns inconsistent with the model assumptions. In this framework, a **true residual plot** refers to the residual plot obtained from the observed data under the fitted regression model. In contrast, **null plots** are synthetic residual plots generated under the null hypothesis H_0, where the linear regression model is correctly specified. These null plots are constructed by generating residuals from the residual rotation distribution (Langsrud 2005), which preserves the structure implied by H_0 (see Section 5 for details).

The goal is to assess whether the true residual plot is visually distinguishable from the null plots. This is typically done using the lineup protocol (Figure 2), where the true residual plot is embedded among multiple null plots. Successful identification of the true residual plot provides evidence against H_0.

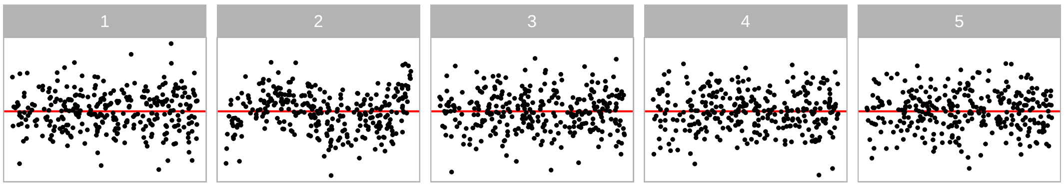

Figure 2. An example lineup embedding the true residual plot among four null plots. In practice, lineups typically include 19 null plots, but a reduced set is shown here for presentation purposes. The null plots are generated via residual rotation to ensure consistency with H_0. Observers who have not previously seen the lineup are asked to identify the plot that appears most different. Under H_0 that the regression model is correctly specified, the true residual plot should be indistinguishable from the null plots, yielding a selection probability of 0.2. A small p-value arises when a substantial proportion of observers select the true residual plot (shown at position 2, exhibiting non-linearity).

The practice of delivering a residual plot as a lineup is generally regarded as a valuable approach. Beyond its application in residual diagnostics, the lineup protocol has been integrated into the analysis of diverse subjects. For instance, Loy and Hofmann (2013; 2014; 2015) illustrated its applicability in diagnosing hierarchical linear models. Additionally, Widen et al. (2016) and Fieberg, Freeman, and Signer (2024) demonstrated its utility in geographical and ecology research respectively, while Krishnan and Hofmann (2021) explored its effectiveness in forensic examinations.

A practical limitation of the lineup protocol lies in its reliance on human judgments (see Li et al. 2024). Unlike conventional statistical tests that can be performed using a statistical software, the lineup protocol requires human evaluation of images. This characteristic makes it less suitable for large-scale applications, given the associated high labor costs and time requirements. There is a substantial need to develop an approach to substitute these human judgement, for example through the automated interpretation of data plots using computer vision.

Efforts to automate plot interpretation date back to early work like Tukey's "Scagnostics" (Tukey and Tukey 1985), which is a set of numerical statistics that summarize features of scatter plots. Wilkinson, Anand, and Grossman (2005) expanded on this work, introducing scagnostics based on computable measures applied to planar proximity graphs. These measures, including, but not limited to, "Outlying", "Skinny", "Stringy", "Straight", "Monotonic", "Skewed", "Clumpy", and "Striated", aimed to characterize outliers, shape, density, trend, coherence and other characteristics of the data. However, such measures may not capture the full range of visual patterns necessary for residual diagnostics (Buja et al. 2009). A more promising direction lies in enabling computers to learn the relevant visual features directly mirroring how humans intuitively interpret plots.

Modern computer vision models rely on deep neural networks with convolutional layers (Fukushima and Miyake 1982). These layers use small, sliding windows to scan the image, performing a dot product to extract local features and patterns. Numerous studies have demonstrated the efficacy of convolutional layers in addressing various vision tasks, including image recognition (Rawat and Wang 2017). Despite the widespread use of computer vision models in fields like computer-aided diagnosis (Lee and Chen 2015), pedestrian detection (Brunetti et al. 2018), and facial recognition (Emami and Suciu 2012), their application in reading data plots remains limited. While some studies have explored the use of computer vision models for tasks such as reading recurrence plots for time series regression (Ojeda, Solano, and Peramo 2020), time series classification (Chu et al. 2019; Hailesilassie 2019; Hatami, Gavet, and Debayle 2018; Zhang et al. 2020), anomaly detection (Chen, Su, and Yang 2020), and pairwise causality analysis (Singh et al. 2017), the application of reading residual plots with computer vision models is a new field of study.

In this paper, we develop computer vision models and integrate them into the residual plot diagnostics workflow, addressing the need for automated visual inference. The model is a regression-based computer vision system that takes a residual plot as input and outputs a non-negative estimated distance $\hat{D}$. Training and test data are generated from a synthetic data model, simulating residual plots under various violation scenarios, with targets defined as customized distances D computed from the underlying data generating process, quantifying the extent of model violations. The model is evaluated on held-out test plots for its ability to estimate $\hat{D}$. In addition, we develop a statistical testing procedure based on $\hat{D}$ for detecting model misspecification, and compare its performance with classical residual diagnostic tests as well as results from a human-subject visual inference experiment using the lineup protocol.

The paper is structured as follows. Section 2 discusses different specifications of the computer vision models. Section 3 introduces the distance measure used to quantify model violations, while Section 4 describes how this measure is estimated using computer vision models, including the model architecture, data generation, and training procedure. Section 5 describes statistical tests based on the estimated distance, and Section 6 presents bootstrap-based methods for assessing the variability of estimated distance and the stability of model misspecification conclusions. Results are reported in Section 7, followed by illustrative applications on example datasets in Section 8. Finally, we conclude with a discussion of the findings and outline directions for future research.

2. Model Specifications

There are various specifications of the computer vision model that can be used to assess residual plots. We discuss these specifications below focusing on the input and the output formats.

2.1. *Input Formats*

Deep learning models are sensitive to input design, and several alternatives are available for encoding residual plots. A simple approach involves feeding vectors of residuals and fitted values into the model. While this format contains all relevant information, the input length varies with sample size, which conflicts with the fixed-shape requirements of modern vision models such as those implemented in TensorFlow (Abadi et al. 2016). Padding can be used to enforce uniform length but risks truncation if inputs exceed the fixed size. Alternatively, fixed-size sampling of residual–fitted value pairs can ensure consistency, though at the cost of some information loss.

Another strategy is to convert residual plots into images. This enables the use of standard convolutional architectures in computer vision such as VGG16 (Simonyan and Zisserman 2014). Image-based inputs offer a richer representation by preserving the joint distribution of residuals and fitted values in a visually interpretable format, helping models more effectively capture spatial structure and local correlations than vector-based formats. To ensure model generalization, it is essential to use residual plots in a consistent style, i.e., same aesthetics, layout, scaling and color schemes.

Using multiple residual plots as input, such as pairs, triplets, or full lineups is also a possible option. For instance, triplet networks (Chopra, Hadsell, and LeCun 2005) can assess visual similarity if we assume null plots are more similar to each other than to the true residual plot, which may help distinguish null plots from true residual plots. However, these architectures introduce complexity in training due to weight sharing and specialized loss functions. We experimented with this setting in a pilot study, and found multiple residual plots led to high input resolution, high computational cost and suboptimal model performance. Balancing accuracy, interpretability, and implementation cost, we adopt a single residual plot in image format as the model input.

2.2. *Output Formats*

Given the input is a single fixed-resolution image, the model output can be defined for various tasks, including binary classification, multiclass classification, and numeric regression.

In the binary case, the outcome may indicate whether the input image is consistent with a null plot, as determined either by (1) the data-generating process or (2) the outcome of a visual test based on human judgment. The first approach models the probability that a residual plot is not a null plot and is a viable option. In contrast, the second relies on experimental data that are often scarce or inconsistent across studies.

Alternatively, a multiclass outcome may classify images into categories such as "contains outliers" vs. "does not contain outliers" or "is non-linear" vs. "not non-linear," among other diagnostic features. While this approach is appealing in theory, it essentially reverts to the traditional testing framework, where tests focus on detecting specific types of model violations in isolation.

A third option is to produce a meaningful and interpretable numerical measure that quantifies the strength of suspicious visual patterns reflecting the extent of model violations, or the difficulty index for identifying whether a residual plot has no issues. However, such measures are often used informally in daily communications but are rarely formalized. For supervised learning, they must be clearly defined and reliably estimable.

In this study, we formulate the output as a continuous measure of deviation from model assumptions. Compared to probability outputs based on binary labels (true residual plots labeled as 1 and null plots as 0), this formulation explicitly encodes the severity of violations and avoids forcing the model to infer such structure solely from the data distribution, which can be inefficient and discards useful prior knowledge.

3. Distance from a Theoretically "Good" Residual Plot

A numerical measure of "difference" or "distance" between plots can take the form of a basic statistical operation on pixels, such as the sum of squared differences. However, direct pixel-to-pixel comparisons are not well suited to residual plots, where the primary interest lies in structural patterns. Alternatively, the distance can be based on established image similarity metrics such as the Structural Similarity Index Measure (Wang et al. 2004), which compares images using contrast, luminance, and structure (related to the average, standard deviation, and correlation of pixel values within a window, respectively). These metrics are designed for general image comparison tasks that differ substantially from evaluating data plots, where only essential plot elements are relevant (Chowdhury et al. 2018). Another direction is to define distance using key scatter plot characteristics, such as those captured by scagnostics (see Section 1), although the functional form must be carefully designed to reflect model violations meaningfully.

In this section, we introduce a distance measure between a true residual plot and a theoretically “good” residual plot. This measure quantifies the divergence between the residual distribution of a given fitted regression model and that of a correctly specified model. Its construction assumes knowledge of the data generating processes for both predictors and response variables. Since these processes are often unknown in practice, Section 4 discusses how the distance can be estimated using a computer vision model, while Section 5 describes a statistical test based on the estimated distance.

3.1. *Residual Distribution*

Consider the classical normal linear regression model, $\boldsymbol{y} = \boldsymbol{X}\boldsymbol{\beta} + \boldsymbol{e}$, where $\boldsymbol{y}$ is the $n \times 1$ response vector, $\boldsymbol{X}$ is the design matrix, $\boldsymbol{\beta}$ is a vector of regression coefficients and $\boldsymbol{e}$ is the vector of random errors. The residual vector is $\hat{\boldsymbol{e}} = \boldsymbol{y} - \hat{\boldsymbol{y}} = \boldsymbol{R}\boldsymbol{e}$, where $\boldsymbol{R} = \boldsymbol{I}_n - \boldsymbol{X}(\boldsymbol{X}^\top\boldsymbol{X})^{-1}\boldsymbol{X}^\top$ is the residual operator, and $\boldsymbol{I}_n$ is the $n \times n$ identity matrix.

Under the classical assumptions $\boldsymbol{e} \sim \mathcal{N}(\boldsymbol{0}_n, \sigma^2\boldsymbol{I}_n)$, the residuals are also Gaussian, since they are a linear transformation of $\boldsymbol{e}$. We refer to this induced distribution as the reference distribution Q, which characterizes the residual behaviour under correct model specification and depends only on $\boldsymbol{X}$.

If $\boldsymbol{X}$ has full column rank k, then $\text{rank}(\boldsymbol{R}) = n - k$, and $\text{cov}(\hat{\boldsymbol{e}}) = \sigma^2\boldsymbol{R}$ is singular. Hence, Q is a degenerate Gaussian supported on an $(n-k)$-dimensional subspace. To obtain a non-degenerate reference distribution required for the distance computation in Section 3.2, we replace $\sigma^2\boldsymbol{R}$ with its diagonal approximation, $\text{diag}(\sigma^2\boldsymbol{R})$, yielding $Q \approx \mathcal{N}\left(\boldsymbol{0}_n, \text{diag}(\sigma^2\boldsymbol{R})\right)$.

The reference distribution Q corresponds to the correct model specification. Under misspecification, the residuals follow a different distribution P. For example, omitted variables correlated with $\boldsymbol{X}$ alter the residual structure, causing P to deviate from Q. Similarly, non-Gaussian error distributions (e.g. multivariate lognormal) induce skewness and heavier tails.

3.2. *Distance of P from Q*

We define the distance of the true residual distribution P from the reference residual distribution Q based on the Kullback–Leibler divergence (Kullback and Leibler 1951):

$$D = \log\left(1 + D_{KL}(P\|Q)\right), \tag{1}$$

where $D_{KL}(P\|Q)$ is defined as

$$D_{KL}(P\|Q) = \int_{\mathbb{R}^n} \log \frac{p(\hat{\boldsymbol{e}})}{q(\hat{\boldsymbol{e}})} p(\hat{\boldsymbol{e}}) d\hat{\boldsymbol{e}}, \tag{2}$$

and $p(\cdot)$ and $q(\cdot)$ are the probability density functions for P and Q, respectively.

This measure was introduced in Li et al. (2024) to quantify the effect size of non-linearity and heteroskedasticity in residual plots. In the presence of model misspecification, if the error structure deviates from $\boldsymbol{e} \sim \mathcal{N}(\boldsymbol{0}_n, \sigma^2\boldsymbol{I}_n)$, for instance with covariance $\boldsymbol{V}$ and omitted higher-order terms $\boldsymbol{Z}\boldsymbol{\beta}_z$ correlated with $\boldsymbol{X}$, the induced residual distribution can be expressed as $P = \mathcal{N}(\boldsymbol{R}\boldsymbol{Z}\boldsymbol{\beta}_z, \text{diag}(\boldsymbol{R}\boldsymbol{V}\boldsymbol{R}))$, where the diagonal approximation ensures non-degenerate covariance matrix. The transformation $D = \log(1+D_{KL})$ compresses large values of D_{KL}, improves numerical stability during estimation (Section 4), and ensures that D is well-defined when $D_{KL} = 0$.

Since P and Q are taken as non-degenerate Gaussian distributions, the KL divergence admits the closed form

$$D_{KL}(P\|Q) = \frac{1}{2}\left(\log\frac{|\boldsymbol{W}|}{|\text{diag}(\boldsymbol{R}\sigma^2)|} - n + \text{tr}(\boldsymbol{W}^{-1}\text{diag}(\boldsymbol{R}\sigma^2)) + \boldsymbol{\mu}_z^\top \boldsymbol{W}^{-1}\boldsymbol{\mu}_z\right), \tag{3}$$

where $\boldsymbol{\mu}_z = \boldsymbol{R}\boldsymbol{Z}\boldsymbol{\beta}_z$, and $\boldsymbol{W} = \text{diag}(\boldsymbol{R}\boldsymbol{V}\boldsymbol{R})$. The assumed error variance σ^2 is set to be $\text{tr}(\boldsymbol{V})/n$, which is the expectation of the estimated variance.

3.3. *Non-normal P*

When the error vector $\boldsymbol{e}$ is non-Gaussian, the induced residual distribution P is generally non-Gaussian and typically unavailable in closed form. Consequently, the Gaussian closed-form expression in Equation 3 is no longer applicable, and the KL divergence must instead be evaluated from its definition in Equation 2. This requires high-dimensional integration over the residual space, rendering direct evaluation impractical for large n.

To obtain a tractable approximation, we assume that the residual components $\{\hat{e}_i\}_{i=1}^n$ are mutually independent, so that $p(\hat{\boldsymbol{e}}) \approx \prod_{i=1}^n p_i(\hat{e}_i)$ and $q(\hat{\boldsymbol{e}}) \approx \prod_{i=1}^n q_i(\hat{e}_i)$. This assumption holds for $Q \approx \mathcal{N}\left(\mathbf{0}_n, \mathrm{diag}(\sigma^2\boldsymbol{R})\right)$, which has independent components, but generally does not hold for P. Nevertheless, it removes the need for high-dimensional integration and reduces the computation to univariate densities, substantially lowering computational cost.

Under this factorisation, the KL divergence decomposes additively:

$$D_{KL}(P\|Q) \approx \sum_{i=1}^{n} \int_{\mathbb{R}} p_i(\hat{e}_i) \log \frac{p_i(\hat{e}_i)}{q_i(\hat{e}_i)} d\hat{e}_i = \sum_{i=1}^{n} \mathbb{E}_{p_i}\left[\log \frac{p_i(\hat{e}_i)}{q_i(\hat{e}_i)}\right],$$

where p_i and q_i denote the marginal densities of $\hat{e}_i$ under P and Q, respectively.

We approximate the expectation via Monte Carlo simulation. Specifically, we generate independent samples $\{\boldsymbol{e}^{(j)}\}_{j=1}^m$ from the error distribution and obtain residual vectors $\boldsymbol{B}_j = \boldsymbol{R}\boldsymbol{e}^{(j)}$. The marginal density p_i is then estimated from $\{B_{ij}\}_{j=1}^m$ using a Gaussian kernel density estimator with bandwidth selected by Silverman's rule of thumb (Silverman 2018), where B_{ij} denotes the i-th component of $\boldsymbol{B}_j$. This yields

$$D_{KL}(P\|Q) \approx \sum_{i=1}^{n} \mathbb{E}_{p_i}\left[\log \frac{p_i(\hat{e}_i)}{q_i(\hat{e}_i)}\right] \approx \sum_{i=1}^{n} \frac{1}{m} \sum_{j=1}^{m} \log \frac{\hat{p}_i(B_{ij})}{q_i(B_{ij})},$$

where $q_i(\cdot)$ denotes the Gaussian reference density with variance $\hat{\sigma}^2 R_{ii}$, with $\hat{\sigma}^2 = \frac{1}{nm-1}\sum_{b\in\mathrm{vec}(\boldsymbol{B})}\left(b-\bar{b}\right)^2$ and $\bar{b} = \frac{1}{nm}\sum_{b\in\mathrm{vec}(\boldsymbol{B})} b$.

4. Distance Estimation Using Computer Vision Models

The distance measure defined in Equation 1 quantifies the difference between the true residual distribution P and the reference distribution Q. However, it can only be computed when the data-generating process is known, which is rarely the case in practice.

To estimate this distance from a residual plot, we define a computer vision estimator

$$\hat{D} = f_{CV}(V_{h\times w}(\boldsymbol{e}, \hat{\boldsymbol{y}}), \boldsymbol{s}),$$

where $V_{h\times w}(\cdot)$ denotes the rendering function that coverts a residuals vs. fitted values plot into an $h \times w$ RGB image, with h and w denoting the image height and width (in pixels), respectively. Here, $\boldsymbol{s}$ is a vector of auxiliary features including scagnostics measures and sample size, and $f_{CV}(\cdot)$ maps the image and auxiliary features to an estimated distance $\hat{D} \in [0, +\infty)$. Details of the model architecture and training procedure are provided in Sections 4.1 and 4.2, respectively.

The auxiliary feature vector $\boldsymbol{s}$ is included to complement information encoded in the residual plot image. While $V_{h\times w}(\cdot)$ captures visual patterns of misspecification, certain attributes such as sample size are not recoverable from the image. In addition, we include scagnostics as useful graph-based summaries of scatter plot structure, which have been shown to be effective descriptors of shape, clustering, and outliers in exploratory data analysis.

The distance estimator $\hat{D}$ quantifies how closely the residuals resemble the reference distribution Q and, after adjusting for sample size effects, can be used as an index of model violation severity (see Appendix D for details). However, since a single residual plot may not fully characterize the underlying residual distribution, $\hat{D}$ is not expected to equal the true distance D. Residual plots generated from the same distribution P may also vary visually, particularly for small n, introducing estimation error that depends on the representativeness of the input plot.

4.1. *Model Architecture*

The architecture of the computer vision model f_{CV} is adapted from the well-established VGG16 architecture (Simonyan and Zisserman 2014). While more recent architectures like ResNet (He et al. 2016) and DenseNet(Huang et al. 2017), have achieved even greater performance, VGG16 remains a solid choice for many applications due to its simplicity and effectiveness. Our decision to use VGG16 aligns with our goal of starting with a proven and straightforward model. Figure 3 illustrates the architecture, while Appendix B provides detailed layer specifications.

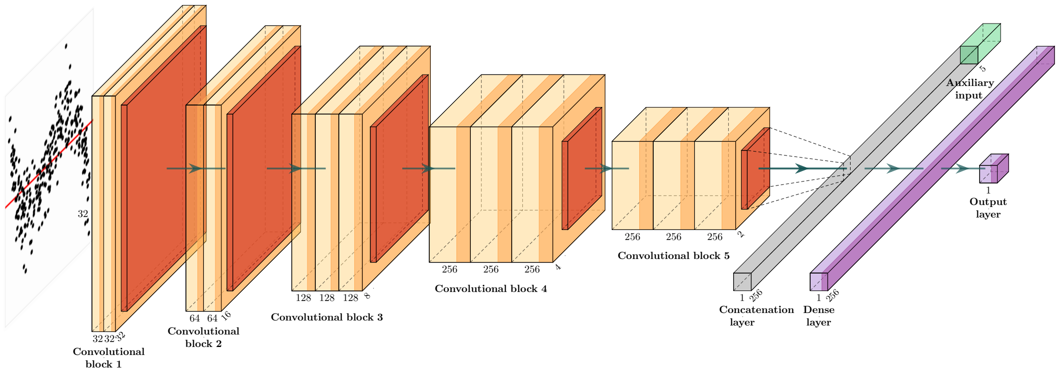

Figure 3. Diagram of the architecture of the optimized computer vision model. Numbers at the bottom of each box show the shape of the output of each layer. The band of each box drawn in a darker color indicates the use of the rectified linear unit activation function. Yellow boxes are 2D convolutional layers, orange boxes are pooling layers, the grey box is the concatenation layer, and the purple boxes are dense layers.

The model accepts residual plots rendered as $h \times w$ RGB images, with $h, w \in \{32, 64, 128\}$, allowing us to study the effect of image resolution on predictive performance while balancing computational cost. Images are first converted to grayscale using the Rec. 601 luma formula (Series 2011), as the data points are plotted in black on a white background. The background also includes a red reference line at $y = 0$, which is preserved under this conversion.

The processed images are used as input to the first convolutional block. The model consists of up to five consecutive convolutional blocks, mirroring the original VGG16 architecture. Within each block, two 3×3 convolutional layers are applied, each followed by batch normalization for training stability and a rectified linear unit (ReLU) activation for non-linearity. The block concludes with a 2×2 max-pooling layer that downsamples the spatial dimensions by taking the maximum over non-overlapping windows for each channel. A dropout layer is then applied at the end of each block to randomly deactivate activations during training, improving regularization.

The final convolutional output is aggregated using either global max-pooling or global average-pooling, producing a two-dimensional feature representation. This feature representation is concatenated with an $n \times 5$ tensor containing the “Monotonic”, “Sparse”, “Splines”, and “Striped” measures computed using the `cassowaryr` R package (Mason et al. 2022), along with the sample size for each residual plot. [1]

The concatenated feature tensor is passed to the final prediction block, which consists of two fully connected layers. The first layer has at least 128 units and is followed by batch normalization and dropout for regularization. The second fully connected layer contains a single unit, followed by a ReLU activation to enforce non-negative outputs, producing the final scalar prediction.

4.2. *Data Generation and Model Training*

To train the computer vision model for estimating $\hat{D}$, we constructed a dataset of paired residual plots and their corresponding D values. In deep learning, synthetic data are commonly used for training vision models when labelled data are scarce, costly, or difficult to obtain reliably (Nikolenko et al. 2021). This is particularly appropriate when the data-generating mechanism is well specified, enabling controlled generation of large-scale examples that encode known structural violations. Accordingly, we generated 80,000 training samples and 8,000 test samples, providing full control over the generative process, exact computation of D, and broad coverage of residual patterns associated with model misspecification.

The simulated data incorporated three common types of residual departures: non-linearity, heteroskedasticity, and non-normality. These effects were induced by fitting a simple linear regression model to data generated from a more flexible underlying process based on Hermite polynomials (Hermite 1864), multiple predictors, and a range of predictor and error distributions. A comprehensive grid of parameter settings was used to produce diverse residual behaviours, including configurations with multiple simultaneous violations.

To ensure uniform coverage across the target space of D, we adopted a bucketed sampling strategy. Specifically, simulated samples were iteratively generated and assigned to one of 50 bins corresponding to quantiles of D, with acceptance controlled to balance representation across difficulty levels. Table 1 summarises the resulting sample

[1] Note: These measures were selected for their reliability and computational efficiency. Other scagnostics were excluded because they occasionally caused R process crashes ($\sim 5\%$) during data preparation due to a bug in the `interp` package (Gebhardt, Bivand, and Sinclair 2023). Although this issue was later fixed at our request, the corrected version was not available during model training, and the full set of features was also deemed too computationally expensive for efficient inference.

Table 1. Number of training and test samples for each model violation scenario, including cases with multiple simultaneous violations. Each sample consists of a residual plot as input and the corresponding distance D as the target. Root mean square error (RMSE) values for the training and test sets are shown for each scenario. Details of the synthetic data generating process used to construct these samples are provided in Appendix A.

	Train		Test	
Violations	#samples	RMSE	#samples	RMSE
no violations	1,546	1.149	155	1.267
non-linearity	22,184	0.625	2,218	0.787
heteroskedasticity	10,826	0.528	1,067	0.602
non-linearity + heteroskedasticity	10,221	0.593	985	0.751
non-normality	10,797	0.279	1,111	0.320
non-linearity + non-normality	8,835	0.472	928	0.600
heteroskedasticity + non-normality	7,870	0.354	819	0.489
non-linearity + heteroskedasticity + non-normality	7,721	0.432	717	0.620

sizes for each violation scenario.

Computer vision model parameters $\boldsymbol{\theta}$ were randomly initialized using the Glorot Uniform method (Glorot and Bengio 2010) and optimized with the Adam optimizer (Kingma and Ba 2014) under a mean squared error objective:

$$\hat{\boldsymbol{\theta}} = \arg\min_{\boldsymbol{\theta}} \frac{1}{n_{\text{train}}} \sum_{i=1}^{n_{\text{train}}} (D_i - f_{\boldsymbol{\theta}}(V_i, \boldsymbol{s}_i))^2,$$

where n_{train} denotes the number of training samples, V_i is the i-th residual plot, and $\boldsymbol{s}_i$ represents the auxiliary tabular features, including four scagnostics measures and sample size.

The model was trained on paired inputs consisting of RGB residual plot images and corresponding D values, with the objective of learning the mapping $\hat{D} = f_{\boldsymbol{\theta}}(V, \boldsymbol{s})$. Training was conducted on the MASSIVE M3 high-performance computing platform (Goscinski et al. 2014) using TensorFlow (Abadi et al. 2016) and Keras (Chollet et al. 2015). Hyperparameters were tuned using Bayesian optimisation via KerasTuner (O'Malley et al. 2019), minimizing validation root mean square error (RMSE) over 100 trials. The search space included dropout rate, batch normalization configuration, input resolution, and the inclusion of auxiliary features.

Further details on the synthetic data-generating process, parameter settings, sampling scheme, example residual plots, and hyperparameter search configuration are provided in Appendix A and Appendix C.

5. Statistical Testing

Under the null hypothesis H_0, corresponding to a correctly specified model, the distance satisfies $D = 0$ because $P = Q$. In practice, however, the estimated distance $\hat{D}$ is generally non-zero, even for null plots, due to sampling variability and imperfections in the computer vision model. Consequently, statistical significance should be assessed relative to the distribution of estimated distances obtained from null plots rather than by the magnitude of $\hat{D}$ alone.

To calibrate $\hat{D}$, we adopt the residual rotation procedure used for null plot generation in Buja et al. (2009). This involves drawing an auxiliary response $\boldsymbol{w}$ from $\mathcal{N}(\mathbf{0}_n, \mathbf{I}_n)$ and compute the projected residuals $\hat{\boldsymbol{e}}_w = \boldsymbol{R}\boldsymbol{w}$. These are then rescaled to match the Euclidean norm of the observed residuals, $\tilde{\boldsymbol{e}}_w = \hat{\boldsymbol{e}}_w \cdot \frac{\|\hat{e}\|}{\|\hat{\boldsymbol{e}}_w\|}$, so that $\|\tilde{\boldsymbol{e}}_w\| = \|\hat{\boldsymbol{e}}\|$. A null plot is then the scatter plot of $(\hat{\boldsymbol{y}}, \tilde{\boldsymbol{e}}_w)$, which under the null hypothesis of a correctly specified model is statistically indistinguishable from the true residual plot $(\hat{\boldsymbol{y}}, \hat{\boldsymbol{e}})$.

Let $\hat{D}$ denote the estimated distance for the true residual plot and let $\{\hat{D}^{(i)}_{null}\}_{i=1}^m$ denote the estimated distances for m null plots generated under H_0. For a lineup containing $m+1$ plots (one true plot and m null plots), the visual inference decision rule is

$$\text{Reject } H_0 \quad \Longleftrightarrow \quad \hat{D} > \max_{1 \le i \le m} \hat{D}^{(i)}_{null}.$$

This corresponds to the standard lineup protocol, where the true residual plot must be identified as the most visually distinct plot.

More generally, the empirical distribution of $\{\hat{D}^{(i)}_{null}\}_{i=1}^m$ provides an estimate of the null distribution of the test statistic. Let $Q_{null}(\tau)$ denote the empirical τ-quantile of this distribution. A level-α test is then given by

$$\text{Reject } H_0 \quad \Longleftrightarrow \quad \hat{D} \ge Q_{null}(1-\alpha).$$

Equivalently, a Monte Carlo p-value can be computed as

$$\hat{p} = \frac{1}{m+1} + \frac{1}{m+1} \sum_{i=1}^{m} \mathbb{I}\left(\hat{D}_{null}^{(i)} \geq \hat{D}\right).$$

This is the standard simulation-based estimator of the tail probability under H_0. For computational efficiency, quantiles of the null distribution may be pre-computed over a range of sample sizes and stored in a lookup table, allowing approximate p-values to be obtained without generating new null plots.

6. Bootstrap Variability Assessment

Bootstrap methods are commonly used in linear regression to assess parameter variability without strong distributional assumptions (Davison and Hinkley 1997; Efron and Tibshirani 1994). This involves resampling the observed data (pairs of responses and predictors) with replacement and refitting the model on each bootstrap sample.

Similarly, this case-resampling bootstrap can be used to assess the variability of the estimated distance $\hat{D}$. For each bootstrap sample $i = 1, \ldots, n_{boot}$, a regression model $M_{boot}^{(i)}$ is refitted using the resampled data, producing a residual plot $V_{boot}^{(i)}$ and a corresponding estimated distance $\hat{D}_{boot}^{(i)}$. The empirical distribution of $\{\hat{D}_{boot}^{(i)}\}_{i=1}^{n_{boot}}$ provides an estimate of the sampling distribution of $\hat{D}$, from which confidence intervals may be constructed.

More generally, each bootstrap model $M_{boot}^{(i)}$ induces its own null distribution of estimated distances. Let $Q_{boot}^{(i)}(\tau)$ denote the τ-quantile of the corresponding null distribution. A level-α test for the i-th bootstrap model is then given by

$$\text{Reject } H_0 \quad \Longleftrightarrow \quad \hat{D}_{boot}^{(i)} \geq Q_{boot}^{(i)}(1-\alpha).$$

The proportion of bootstrap models for which H_0 is rejected,

$$\frac{1}{n_{boot}} \sum_{i=1}^{n_{boot}} \mathbb{I}\left(\hat{D}_{boot}^{(i)} \geq Q_{boot}^{(i)}(1-\alpha)\right),$$

estimates how often the fitted regression model would be judged misspecified under repeated sampling from the same data-generating process.

This procedure is computationally expensive because it requires evaluating $n_{boot}(m+1)$ residual plots, where m denotes the number of null plots per bootstrap sample. In practice, the bootstrap-specific critical values $Q^{(i)}_{boot}(1-\alpha)$ may be approximated by the corresponding quantile $Q_{null}(1-\alpha)$ from the original null distribution, substantially reducing the computational burden.

7. Results

7.1. *Model Performance*

Table 2 summarizes the test performance of the optimized models across three input sizes, using 80,000 training samples and 8,000 test samples generated as described in Section 4.2. The 32×32 model consistently outperformed others, with a mean absolute error of approximately 0.43, a negligible deviation given the typical D range of 0 to 7. High R^2 values also indicated strong linear correlation between predictions and targets. Based on these results, we used the best-performing 32×32 model for subsequent analyses.

Figure 4 shows a hexagonal heatmap of $D - \hat{D}$ versus D. Smoothing curves (fitted by generalized additive models, Hastie 2017) reveal that all models performed well for $1.5 < D < 6$, where no structural issues were observed. Over-prediction occurred when $D < 1.5$, and under-prediction primarily when $\hat{D} > 6$.

For null plots ($D = 0$), over-prediction was expected (see Section 5), but not under-prediction. Therefore, we analysed the relationship between residuals and all the factors involved in the data generating process. We found that most deviations stemmed from non-linearity and the presence of a second predictor in the synthetic data model, where small error variances caused under-predictions and large error variances caused over-prediction, as illustrated in Figure 5.

To investigate further, we stratified the train and test set by violation type and re-evaluated performance (Table 1). It was found that null plots had the poorest metrics, while non-normality cases were easiest to predict, with RMSE around 0.3. Non-

Table 2. The test performance of three optimized models with different input sizes. The metrics are computed by comparing the estimated distance $\hat{D}$ (model output) and the target distance D generated from the synthetic data model. RMSE represents the root mean squared error; R^2 is the squared correlation between the two quantities; MAE denotes the mean absolute error; and Huber loss refers to the average Huber loss computed over the test set.

	RMSE	R^2	MAE	Huber loss
32×32	0.660	0.901	0.434	0.18
64×64	0.674	0.897	0.438	0.19
128×128	0.692	0.892	0.460	0.20

linearity (RMSE ≈ 0.8) was harder to assess than heteroskedasticity (RMSE ≈ 0.6) or non-normality (RMSE ≈ 0.3). Plots with multiple violations showed intermediate performance.

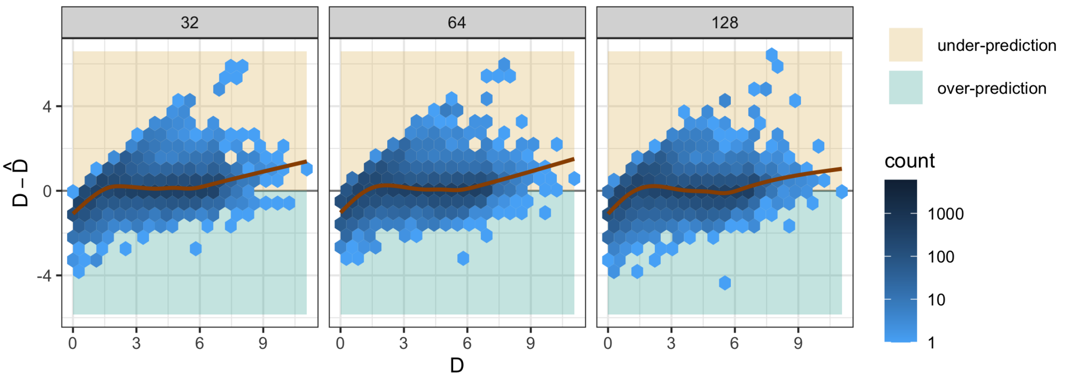

Figure 4. Hexagonal heatmap for difference in D and $\hat{D}$ vs D on test data for three optimized models with different input sizes. The brown lines are smoothing curves produced by fitting generalized additive models. Over-prediction and under-prediction can be observed for small D and large D respectively.

7.2. *Comparison with Human Visual Inference and Conventional Tests*

7.2.1. Overview of the Human Subject Experiment

In order to check the validity of the proposed computer vision model, residual plots presented in the human subject experiment conducted by Li et al. (2024) was assessed. The study collected 7,974 responses across 1,152 lineups, each containing one randomly positioned true residual plot and 19 null plots. Of these, 24 were attention-check lineups with highly obvious patterns, 36 were null lineups containing only null plots, 279 involved uniformly distributed predictors and were evaluated by 11 partici-

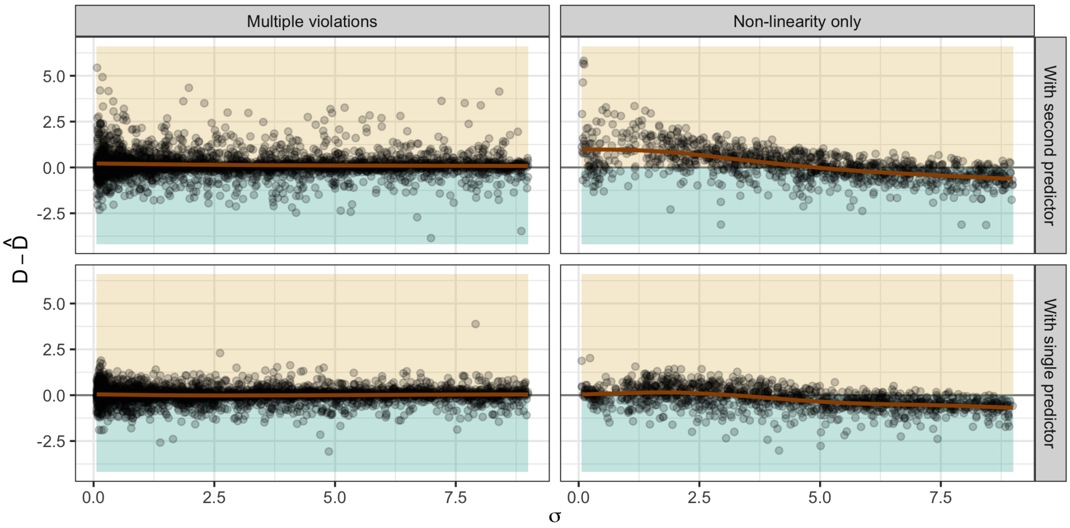

Figure 5. Scatter plots for difference in D and $\hat{D}$ vs σ on test data for the 32×32 optimized model. The data is grouped by whether the regression has only non-linearity violation, and whether it includes a second predictor in the regression formula. The brown lines are smoothing curves produced by fitting generalized additive models. Under-prediction mainly occurs when the data-generating process has small σ, a second predictor, and only non-linearity as the model violation.

pants each, and the remaining 813 involved discrete, skewed, or normally distributed predictors and were evaluated by 5 participants each. Attention-check and null lineups are excluded from the present analysis.

Participants were recruited through Prolific. Most were not experts in residual plots, and 56.7% reported no prior experience with studies involving data visualisations. The sample was approximately gender-balanced, more than half held a diploma or bachelor's degree, and almost all participants were between 18 and 39 years of age. Consequently, the results primarily reflect the ability of non-experts to distinguish true residual plots from null plots based on visual differences rather than specialised statistical training.

The residual plots in Li et al. (2024) were generated from a special case of the synthetic data model used in this study. Model violations were introduced independently, so non-linearity and heteroskedasticity did not co-occur within the same lineup. In addition, the experimental design did not consider non-normality or multiple predictors.

Table 3. The performance of the 32×32 model on the data used in the human subject experiment.

Violation	RMSE	R^2	MAE	Huber loss
heteroskedasticity	0.721	0.852	0.553	0.235
non-linearity	0.738	0.770	0.566	0.246

Table 4. Summary of the comparison of decisions made by computer vision model with decisions made by conventional tests and visual tests conducted by human.

Violations	#Samples	#Agreements	Agreement rate
Compared with conventional tests			
heteroskedasticity	540	464	0.8593
non-linearity	576	459	0.7969
Compared with visual tests conducted by human			
heteroskedasticity	540	367	0.6796
non-linearity	576	385	0.6684

7.2.2. Model Performance on the Human-evaluated Data

Each lineup in Li et al. (2024) contains one true residual plot and 19 null plots. While the true residual plot's distance D depends on the data-generating process, null plots have $D = 0$. We used our optimized computer vision model to estimate D for all plots. To ensure fair comparison, we reject H_0 if the true residual plot has the highest estimated distance. Conventional tests, including the Ramsey RESET (Ramsey 1969) for non-linearity and the Breusch-Pagan test (Breusch and Pagan 1979) for heteroskedasticity, were also applied to the same data.

Table 3 reports performance metrics for $\hat{D}$ on true residual plots. While the performance is slightly worse than on the test data provided in Table 2, the mean absolute error remains low and the correlation between predictions and true values remains high. Consistent with results in Table 1, non-linearity cases are harder to predict than heteroskedasticity cases.

Table 4 compares the model's rejection decisions with those of conventional tests and human visual tests. Agreement rates are 85.95% for heteroskedasticity cases and 79.69% for non-linearity cases, higher than those observed for human visual tests (67.96% and 66.84%, respectively). This suggests that the model aligns more closely with the best available conventional tests. However, as Figure 6 shows, the model does not always reject when conventional tests do, indicating a lower sensitivity to model violations.

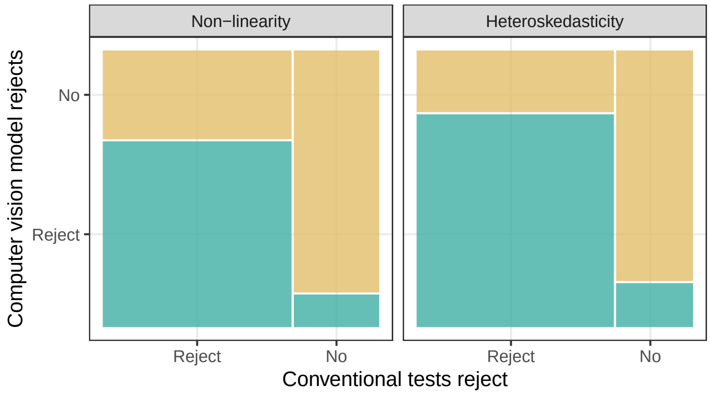

Figure 6. Rejection rate (p-value ≤ 0.05) of computer vision models conditional on conventional tests on non-linearity (left) and heteroskedasticity (right) lineups displayed using a mosaic plot. When the conventional test fails to reject, the computer vision mostly fails to reject the same plot as well as indicated by the height of the top right yellow rectangle, but there are non negligible amount of plots where the conventional test rejects but the computer vision model fails to reject as indicated by the height of the top left yellow rectangle.

To further illustrate these patterns, Figure 7 presents parallel coordinate plots comparing decisions from human visual tests, the computer vision model, and conventional tests. All three agree in about 50% of cases. When humans do not reject, conventional tests reject in about half of those cases, whereas the model rejects in only 25%. Notably, the model rarely rejects when both humans and conventional tests do not, suggesting that while it is more sensitive than humans, its rejection behavior is more aligned with human judgment than with conventional tests.

Figure 8 plots rejection decisions against the true distance D. Compared to conventional tests, the computer vision model rejects fewer cases when $D < 4$, but a similar number of cases when $D > 4$. This indicates that the model is less sensitive to minor deviations from model assumptions, while remaining comparably sensitive to more substantial violations. Visual tests are the least sensitive overall, showing few rejections for $D < 2$, and requiring a higher threshold ($D > 5$) for near-universal rejection, compared to $D > 4$ for both the model and conventional tests.

Finally, the power curves in Figure 8 are fitted using logistic regression without intercepts and with an offset of $\log(0.05/0.95)$ to ensure a 5% rejection rate under H_0. The model's rejection curve consistently lies between those of conventional and human

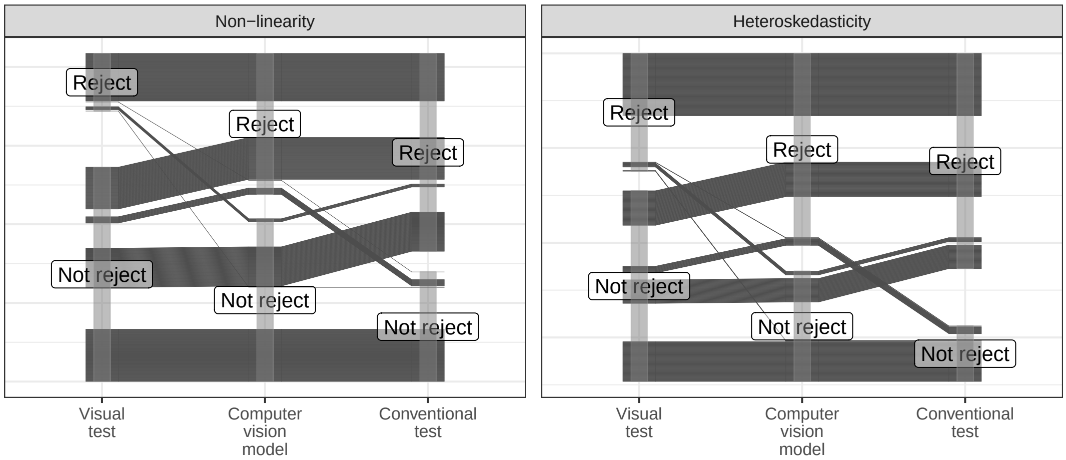

Figure 7. Parallel coordinate plots of decisions made by computer vision model, conventional tests and visual tests made by human. All three agree in around 50% of cases. The model rejects less often than conventional tests when humans do not, and rarely rejects when both others do not, indicating closer alignment with human judgment.

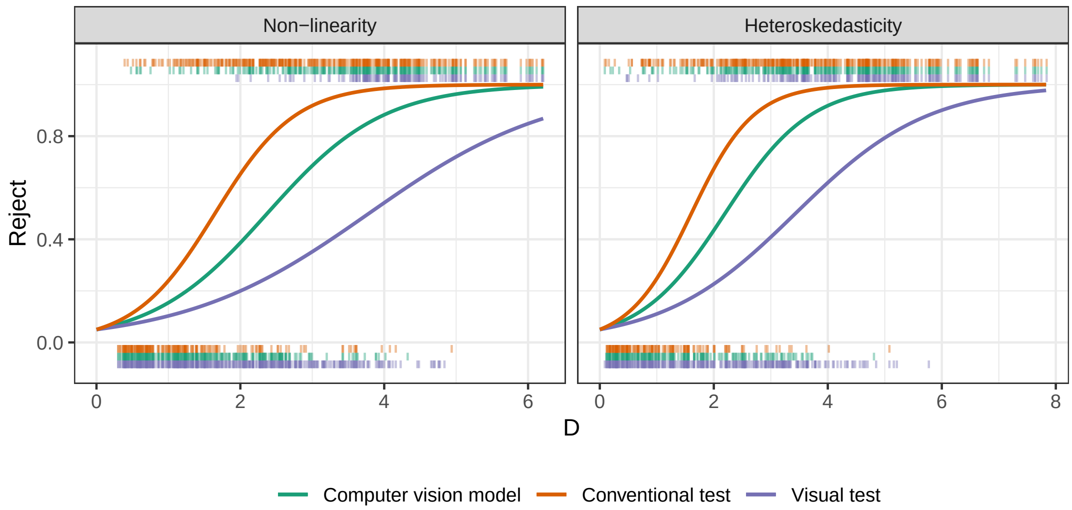

Figure 8. Comparison of power of visual tests, conventional tests and the computer vision model. Marks along the x-axis at the bottom of the plot represent rejections made by each type of test. Marks at the top of the plot represent acceptances. Power curves are fitted by logistic regression models with no intercept but an offset equals to $\log(0.05/0.95)$. The model is less sensitive than conventional tests for small D but similarly sensitive for large D. Visual tests are least sensitive overall. The model's curve lies between those of conventional and visual tests.

visual tests, indicating potential to further align its decision boundary with human visual assessment.

7.2.3. Adjusted δ-difference

In the experiment conducted by Li et al. (2024), participants could select multiple plots per lineup. To reflect detection confidence, a weighted detection rate was computed for each lineup: selecting zero plots yielded a weight of 0.05; otherwise, if the true residual plot was selected, its weight was the inverse of the number of selections.

To evaluate the proposed distance measure, we could examine the relationship between the weighted detection rate and the δ-difference statistic from Chowdhury et al. (2018):

$$\delta = \bar{d}_{\text{true}} - \max_j \left(\bar{d}^{(j)}_{\text{null}}\right) \quad \text{for } j = 1, ..., m-1,$$

where $\bar{d}_{\text{true}}$ is the mean distance between the true residual plot and all null plots, and $\bar{d}^{(j)}_{\text{null}}$ is the mean distance from null plot j to other nulls. These averages help address asymmetries in pairwise distances.

However, our method only compares each plot to a theoretically "good" residual plot, making the original formulation inapplicable. Moreover, since $D_{\text{null}} = 0$ by definition and D can not be derived from an image, we instead focus on $\hat{D}$, the model-estimated distance. This leads to the adjusted δ-difference:

$$\delta_{\text{adj}} = \hat{D} - \max_j \left(\hat{D}^{(j)}_{\text{null}}\right) \quad \text{for } j = 1, ..., m-1,$$

where $\hat{D}^{(j)}_{\text{null}}$ is the estimated distance for the j-th null plot, and m is the number of plots in a lineup.

Figure 9 displays the scatter plot of the weighted detection rate against δ_{adj}. It shows that weighted detection generally increases with δ_{adj}, especially when it is positive. This suggests that the differences in $\hat{D}$ capture some aspect of the visual separability between the true and null plots as perceived by humans, with larger positive differences generally indicating greater distinctiveness. However, the considerable variability in detection rates for similar δ_{adj} values implies that $\hat{D}$ does not fully account for all factors influencing human perception, and the alignment remains imperfect. A negative δ_{adj} suggests some null plots may appear more visually distinctive to humans than the true residual plot. In some instances, the weighted detection rate is close to one despite

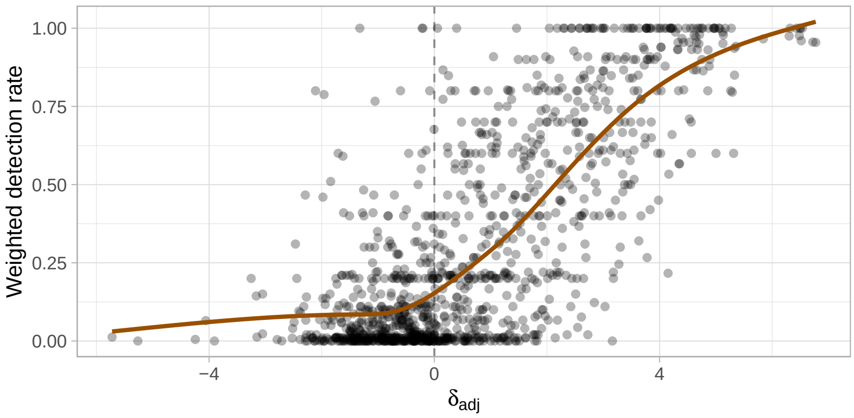

Figure 9. A weighted detection rate vs adjusted δ-difference plot. The brown line is smoothing curve produced by fitting generalized additive models. Detection generally increases with positive δ_{adj}, but variability and exceptions highlight the distance measure's imperfect alignment with human perception.

a negative δ_{adj}. This discrepancy implies that the distance measure may not perfectly reflect actual human behavior.

8. Examples

In this section, we present the performance of the trained computer vision model on three example datasets, including the dataset associated with the residual plot displaying a "left-triangle" shape, the Boston housing dataset (Harrison Jr and Rubinfeld 1978), and the "dino" datasets from the from the `datasauRus` package (Davies, Locke, and D'Agostino McGowan 2022).

In the first case, both the model and human inspection correctly avoid rejecting H_0 when it is true, contrary to conventional tests, highlighting the necessity of visual examination. The second case shows a clear model violation detected by all tests, demonstrating the model's practical use for less intricate tasks. The third case involves an unusual deviation where the model and humans reject H_0, but some conventional tests do not, illustrating the benefits of visual-based decision-making.

8.1. *Left-triangle*

In Section 1, we introduced a residual plot (Figure 10A) where the "left-triangle" shape could be misinterpreted as evidence of heteroskedasticity. Although the residuals are drawn from a correctly specified model, the Breusch-Pagan test rejects H_0 ($p =$ 0.046). This visual artifact stems from the skewed distribution of fitted values, as later illustrated in the lineup in Figure 13A, where similar "left-triangle" patterns appear across all residual plots. Since the true plot (at position 10) is visually indistinguishable from the null plots, a visual test would not reject H_0.

The computer vision model yields the same conclusion from the lineup. As shown in Figure 10C, the estimated distance $\hat{D}$ falls well below the 95% quantile of the null distribution, with a p-value of 0.33. The bootstrapped distribution further suggests that rejection of the fitted model is highly unlikely. Together, the visual and model-based tests do not reject H_0, in contrast to the Breusch-Pagan test, which lacks the context provided by null plots and may be misled by visual artifacts.

To further interpret the model's output, the attention map in Figure 10B, computed as the gradient of the output with respect to the grayscale input, shows that the top-right and bottom-right corners of the residual plot have the most influence, corresponding to the triangle's vertices. In addition, a principal component analysis (PCA) of the 256 features extracted by the model's global pooling layer (Figure 10D) shows that null and bootstrapped plots cluster together in the space defined by the first two principal components, and the true residual plot is covered by the cluster. This overlap reinforces the conclusion that the true plot is visually indistinct, aligning with our interpretation of the lineup.

8.2. *Boston Housing*

The Boston housing dataset, originally published by Harrison Jr and Rubinfeld (1978), provides insights into housing in the Boston, Massachusetts area. For illustration, we use a reduced Kaggle version containing 489 observations and four variables: average number of rooms per dwelling (RM), percentage of lower status population (LSTAT), pupil-teacher ratio by town (PTRATIO), and median home value (MEDV). We fit a linear regression model with MEDV as the response and the remaining variables as predictors, focusing on detecting non-linearity, because the relationships between RM and MEDV or LSTAT and MEDV are non-linear.

Figure 11 summarizes the model evaluation. The residual plot (Figure 11A) reveals a

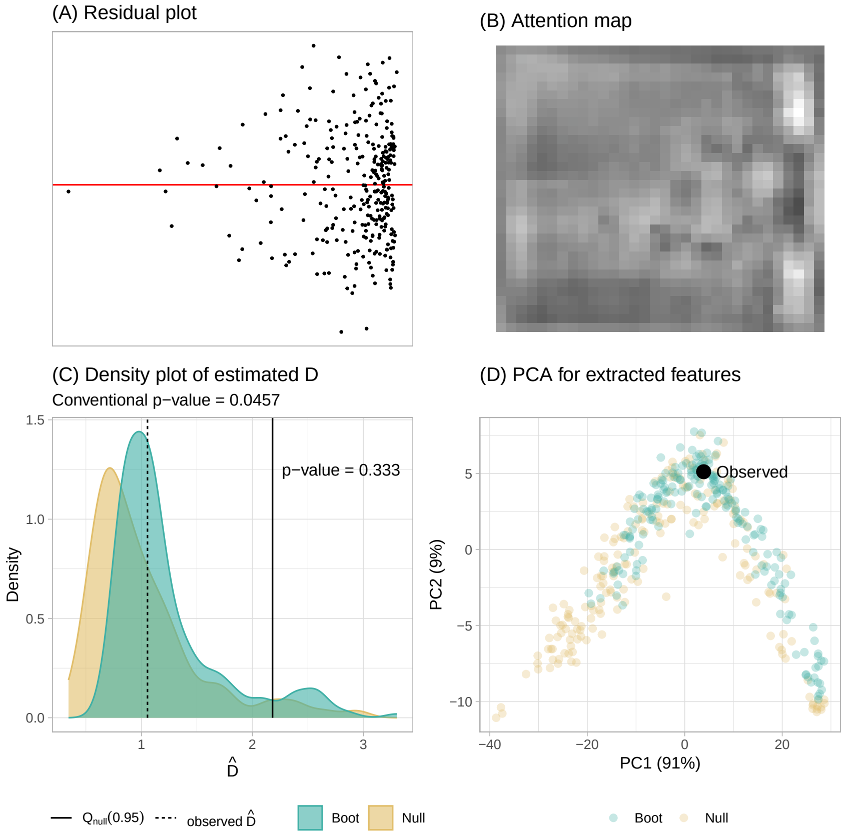

Figure 10. A summary of the residual plot assessment evaluated on 200 null plots and 200 bootstrapped plots. (A) The true residual plot exhibiting a "left-triangle" shape. (B) The attention map highlights the top-right and bottom-right corners of the residual plot as the most influential. (C) The density plot shows estimated distances for null (yellow) and bootstrapped (green) plots. The fitted model is not rejected because $\hat{D} < Q_{null}(0.95)$. (D) Null and bootstrapped plots cluster together in the space defined by the first two principal components of the global pooling layer. The cluster also covers the true residual plot.

distinct "U"-shaped pattern, suggesting non-linearity. The RESET test strongly rejects H_0 with a very small p-value, and the computer vision model yields a large estimated distance $\hat{D}$, well above the 95% quantile of the null distribution (p-value $= 0.00498$). The bootstrapped distribution also shows widespread rejection, indicating the model is likely misspecified. The attention map (Figure 11B) highlights the central region of the residual plot, corresponding to the turning point of the "U", as most influential. In the PCA projection (Figure 11D), bootstrapped and null plots form two distinct clusters, emphasizing the visual separability. This aligns with the lineup in Figure 13B, where the true plot stands out clearly. A human visual test would also likely reject H_0 in this case.

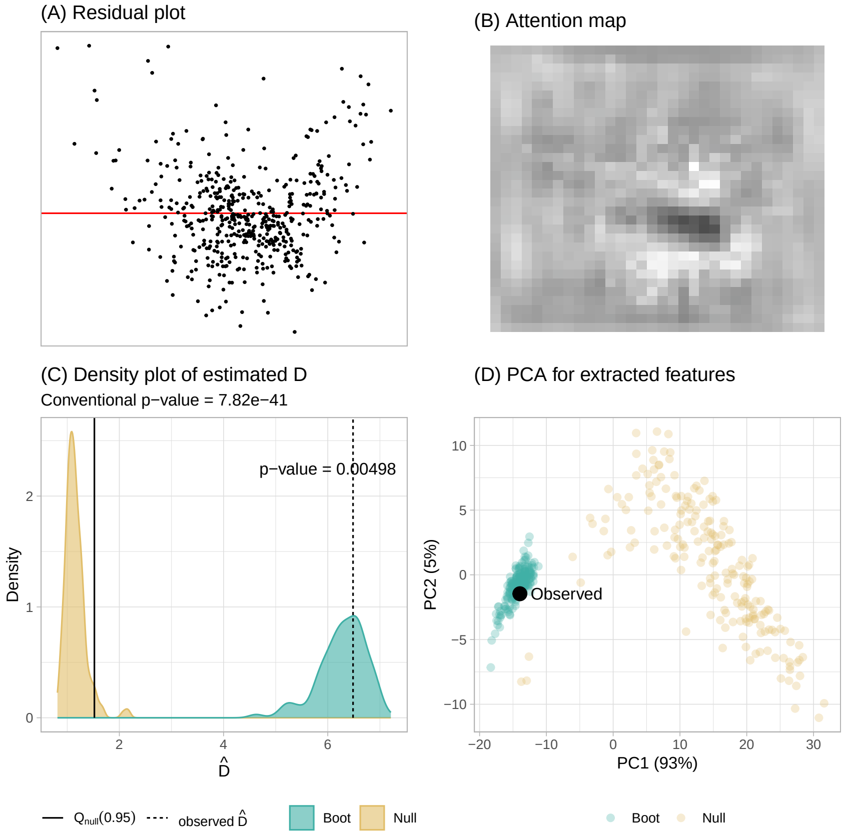

Figure 11. A summary of the residual plot assessment for the Boston housing fitted model evaluated on 200 null plots and 200 bootstrapped plots. (A) The true residual plot exhibiting a "U" shape. (B) The attention map highlights the central region of the "U" shape as the most influential part of the residual plot. (C) The density plot shows estimated distances for null (yellow) and bootstrapped (green) plots. The fitted model is rejected because $\hat{D} \geq Q_{null}(0.95)$. (D) Bootstrapped and null plots form distinct clusters in the space of the first two principal components from the global pooling layer, highlighting their visual separability.

8.3. *DatasauRus*

Beyond detecting common issues like non-linearity, heteroskedasticity, and non-normality, the computer vision model can also identify unusual visual artifacts, such as shapes resembling real-world objects, as long as such patterns do not appear in null plots. These cases are challenging to categorize using conventional model diagnostics, so we compare the model's output with results from the RESET test, Breusch-Pagan test, and Shapiro-Wilk test (Shapiro and Wilk 1965).

The "dino" dataset from the `datasauRus` R package illustrates this situation. It contains only two variables, x and y, but fitting a regression model produces a residual plot that clearly resembles a dinosaur (Figure 12A). This pattern is visually distinctive in the lineup shown in Figure 13C, where a human visual test would undoubtedly reject

H_0.

Consistent with this, the computer vision model assigns a distance $\hat{D}$ above the 95% quantile of the null distribution (p-value $= 0.00498$), leading to rejection of H_0. The bootstrapped distribution further supports this, with most fitted models also being rejected. In contrast, the RESET and Breusch-Pagan tests yield p-values above 0.3, suggesting no violation. Only the Shapiro-Wilk test rejects normality with a small p-value.

Crucially, the attention map in Figure 12B highlights the dinosaur shape, indicating that the model's decision is driven by human-perceptible features. The model appears to capture contours and outlines in a manner similar to human visual interpretation. In the PCA plot (Figure 12D), the cluster of bootstrapped plots is positioned at the corner of the cluster of null plots, isolated from the cluster of null plots.

In practice, such visual artifacts would be difficult to detect without inspecting the residual plot directly. Furthermore, analysts do not always test for normality, and even when violations are found, they are often deemed acceptable due to the robustness of linear regression under quasi-likelihood theory. This example highlights the value of visual diagnostics and the unique role of the proposed computer vision model in supporting them.

9. Limitations and Future Work

While the computer vision model performs well under the synthetic data generation scheme and across the three illustrative examples, this study has several limitations that point to directions for future work.

First, the proposed distance measure assumes the true model follows a classical normal linear regression framework, which may be restrictive. Although we do not explore relaxation of this assumption here, future work could extend the approach to other types of regression models. One possibility is to define a distance measure tailored to each model class and re-train the computer vision model accordingly. To reduce computational cost, the convolutional blocks from our current model could be reused via transfer learning, as it is already effective at capturing visual features. Alternatively,

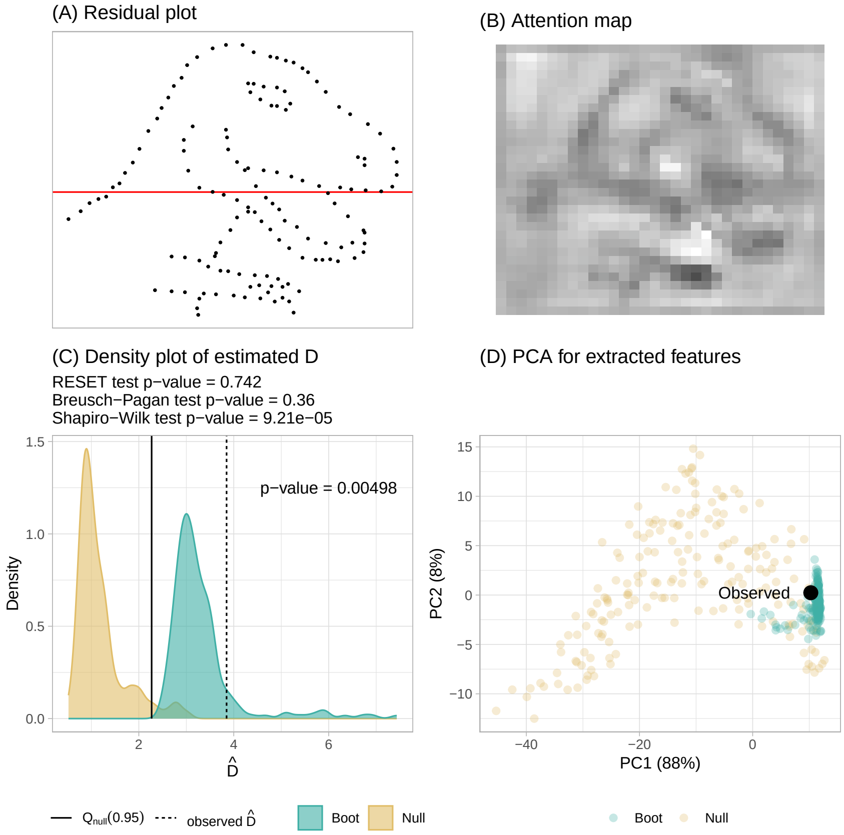

Figure 12. A summary of the residual plot assessment for the datasauRus fitted model evaluated on 200 null plots and 200 bootstrapped plots. (A) The residual plot exhibits a "dinosaur" shape. (B) The attention map highlights the "dinosaur" shape, indicating the model's decision is driven by human-recognizable features.(C) The density plot shows estimated distances for null (yellow) and bootstrapped (green) plots. The fitted model is rejected because $\hat{D} \geq Q_{null}(0.95)$. (D) The bootstrapped plots cluster at the corner of the null plot cluster, yet remain isolated in the space defined by the first two principal components of the global pooling layer.

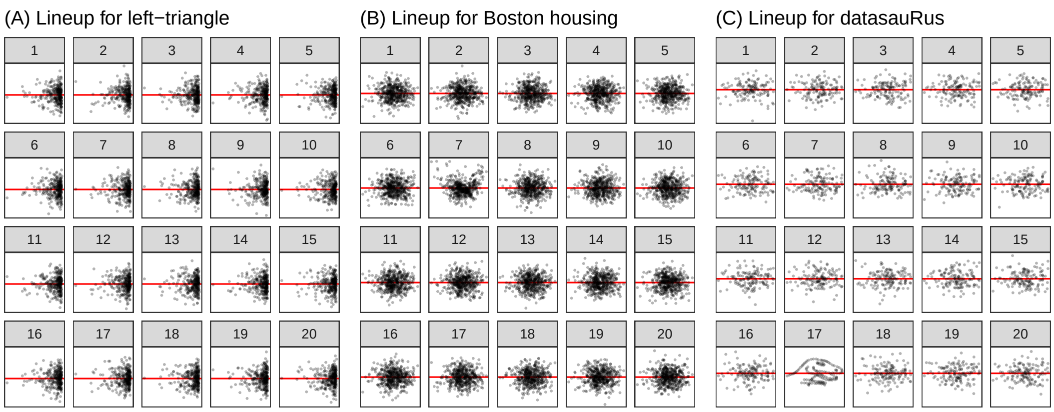

Figure 13. Lineups of residual plots for the "left-triangle", "Boston housing", and "datasauRus" datasets. (A) True plot at position 10; no clear visual difference. (B) True plot at position 7; "U"-shape clearly stands out. (C) True plot at position 17; distinctly artificial and easily identifiable.

transforming residuals to approximate normality and constant variance could allow for meaningful comparisons under our current distance measure. However, if raw residuals are used, the interpretation remains valid only when the visual differences between the true and null plots are detectable by the proposed distance measure.

Second, this study focused solely on the standard residuals vs. fitted values plot, leaving out other common diagnostic plots such as residuals vs. predictors and Q-Q plots. These alternative plots may offer additional information and could be incorporated into a broader visual testing framework in future research.

Finally, we chose a relatively simple computer vision architecture to establish a baseline for automated visual inference. While the current model shows promise, there is room to improve its alignment with human judgment. Achieving this may require external data from surveys or controlled experiments to better understand how human observers interpret residual plots and how these judgments differ from the model's outputs.

10. Conclusions

In this paper, we have introduced a distance measure based on the Kullback-Leibler divergence to quantify disparity between the residual distribution of a fitted classical normal linear regression model and the reference distribution under correct specification. This measure captures the extent of model violations and forms the basis of a computer vision model that estimates the distance using residual plots as input.

The estimated distance supports a formal statistical testing procedure by comparing the true residual plot against null plots generated from the fitted model. Through bootstrapping and model refitting, we also evaluate how often a correctly specified model would be flagged as misspecified under repeated sampling.

The trained computer vision model performs well on both training and test data, though its performance is somewhat lower for non-linear patterns. Statistical tests based on the model's predicted distance show lower sensitivity than conventional tests but higher sensitivity than human visual tests. Importantly, the predicted distance broadly aligns with the strength of visual signals perceived by humans, though further improvements are possible.

We demonstrate the method across several scenarios, highlighting its effectiveness and its conceptual alignment with visual inference. Both visual and distance-based tests act as omnibus procedures, detecting a wide range of model violations collectively. In contrast, conventional tests such as RESET and Breusch-Pagan target specific assumptions, making them less flexible and potentially harder to combine without adjusting for multiple testing.

Overall, our approach offers a practical tool to support regression diagnostics by reducing the manual effort required for residual plot inspection. While we recommend analysts to continue reading residual plots whenever feasible, our method can augment the process, either as an automated screening tool or a supplement to human judgment.

Supplementary Materials

Appendix: The appendix includes more details about the synthetic data generation process, neural network layers used in the work, model training, and model violation index. (appendix.pdf, Portable Document Format file)

Acknowledgement

These `R` packages were used for the work: `tidyverse` (Wickham et al. 2019), `lmtest` (Zeileis and Hothorn 2002), `mpoly` (Kahle 2013), `ggmosaic` (Jeppson, Hofmann, and Cook 2021), `kableExtra` (Zhu 2021), `patchwork` (Pedersen 2022), `glue` (Hester and Bryan 2022), `ggpcp` (Hofmann, VanderPlas, and Ge 2022), `here` (Müller 2020), and `reticulate` (Ushey, Allaire, and Tang 2024).

The article was created with R packages `rticles` (Allaire et al. 2022), `knitr` (Xie 2014) and `rmarkdown` (Xie, Dervieux, and Riederer 2020). The project's GitHub repository (`https://github.com/TengMCing/auto_residual_reading_paper`) contains all materials required to reproduce this article.

We acknowledge the use of ChatGPT (OpenAI, GPT-4, accessed July 2025) to improve the clarity and fluency of the manuscript text. All content was reviewed and

verified by the authors to ensure accuracy and integrity.

Data Availability Statement

The data that support the findings of this study are openly available in the project's GitHub repository (`https://github.com/TengMCing/auto_residual_reading_paper`).

Disclosure Statement

No potential conflict of interest was reported by the author(s).

References

Abadi, Martín, Ashish Agarwal, Paul Barham, Eugene Brevdo, Zhifeng Chen, Craig Citro, Greg S Corrado, et al. 2016. "Tensorflow: Large-scale machine learning on heterogeneous distributed systems." *arXiv preprint arXiv:1603.04467* .

Allaire, JJ, Yihui Xie, Christophe Dervieux, R Foundation, Hadley Wickham, Journal of Statistical Software, Ramnath Vaidyanathan, et al. 2022. *rticles: Article formats for R Markdown.* R package version 0.24, `https://CRAN.R-project.org/package=rticles`.

Belsley, David A, Edwin Kuh, and Roy E Welsch. 1980. *Regression diagnostics: Identifying influential data and sources of collinearity.* John Wiley & Sons.

Breusch, Trevor S, and Adrian R Pagan. 1979. "A simple test for heteroscedasticity and random coefficient variation." *Econometrica: Journal of the Econometric Society* 1287–1294.

Brunetti, Antonio, Domenico Buongiorno, Gianpaolo Francesco Trotta, and Vitoantonio Bevilacqua. 2018. "Computer vision and deep learning techniques for pedestrian detection and tracking: A survey." *Neurocomputing* 300: 17–33.

Buja, Andreas, Dianne Cook, Heike Hofmann, Michael Lawrence, Eun-Kyung Lee, Deborah F Swayne, and Hadley Wickham. 2009. "Statistical inference for exploratory data analysis and model diagnostics." *Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences* 367 (1906): 4361–4383.

Chen, Yun, Shijie Su, and Hui Yang. 2020. "Convolutional neural network analysis of recurrence plots for anomaly detection." *International Journal of Bifurcation and Chaos* 30 (01): 2050002.

Chollet, François, et al. 2015. "Keras." `https://keras.io`.

Chopra, Sumit, Raia Hadsell, and Yann LeCun. 2005. "Learning a similarity metric discriminatively, with application to face verification." In *2005 IEEE computer society conference on computer vision and pattern recognition (CVPR'05)*, Vol. 1, 539–546. IEEE.

Chowdhury, Niladri Roy, Dianne Cook, Heike Hofmann, and Mahbubul Majumder. 2018. "Measuring lineup difficulty by matching distance metrics with subject choices in crowd-sourced data." *Journal of Computational and Graphical Statistics* 27 (1): 132–145.

Chu, Hongyang, Xinwei Liao, Peng Dong, Zhiming Chen, Xiaoliang Zhao, and Jiandong Zou. 2019. "An automatic classification method of well testing plot based on convolutional neural network (CNN)." *Energies* 12 (15): 2846.

Cook, R Dennis, and Sanford Weisberg. 1982. *Residuals and influence in regression.* New York: Chapman and Hall.

Davies, Rhian, Steph Locke, and Lucy D'Agostino McGowan. 2022. *datasauRus: Datasets from the Datasaurus Dozen.* R package version 0.1.6, `https://CRAN.R-project.org/package=datasauRus`.

Davison, Anthony Christopher, and David Victor Hinkley. 1997. *Bootstrap methods and their application.* Cambridge university press.

Efron, Bradley, and Robert J Tibshirani. 1994. *An introduction to the bootstrap.* Chapman and Hall/CRC.

Emami, Shervin, and Valentin Petrut Suciu. 2012. "Facial recognition using OpenCV." *Journal of Mobile, Embedded and Distributed Systems* 4 (1): 38–43.

Fieberg, John, Smith Freeman, and Johannes Signer. 2024. "Using lineups to evaluate goodness of fit of animal movement models." *Methods in Ecology and Evolution* .

Fukushima, Kunihiko, and Sei Miyake. 1982. "Neocognitron: A new algorithm for pattern recognition tolerant of deformations and shifts in position." *Pattern recognition* 15 (6): 455–469.

Gebhardt, Albrecht, Roger Bivand, and David Sinclair. 2023. *interp: Interpolation Methods.* R package version 1.1-5, `https://CRAN.R-project.org/package=interp`.

Glorot, Xavier, and Yoshua Bengio. 2010. "Understanding the difficulty of training deep feed-forward neural networks." In *Proceedings of the thirteenth international conference on artificial intelligence and statistics*, 249–256. JMLR Workshop and Conference Proceedings.

Goscinski, Wojtek J, Paul McIntosh, Ulrich Felzmann, Anton Maksimenko, Christopher J Hall, Timur Gureyev, Darren Thompson, et al. 2014. "The multi-modal Australian ScienceS Imaging and Visualization Environment (MASSIVE) high performance computing infrastructure: applications in neuroscience and neuroinformatics research." *Frontiers in Neuroinformatics* 8: 30.

Hailesilassie, Tameru. 2019. "Financial Market Prediction Using Recurrence Plot and Convolutional Neural Network." .

Harrison Jr, David, and Daniel L Rubinfeld. 1978. "Hedonic housing prices and the demand for clean air." *Journal of environmental economics and management* 5 (1): 81–102.

Hastie, Trevor J. 2017. "Generalized additive models." In *Statistical models in S*, 249–307. Routledge.

Hatami, Nima, Yann Gavet, and Johan Debayle. 2018. "Classification of time-series images using deep convolutional neural networks." In *Tenth international conference on machine vision (ICMV 2017)*, Vol. 10696, 242–249. SPIE.

He, Kaiming, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. "Deep residual learning for image recognition." In *Proceedings of the IEEE conference on computer vision and pattern recognition*, 770–778.

Hermite, M. 1864. *Sur un nouveau développement en série des fonctions*. Imprimerie de Gauthier-Villars.

Hester, Jim, and Jennifer Bryan. 2022. *glue: Interpreted String Literals*. R package version 1.6.2, `https://CRAN.R-project.org/package=glue`.

Hofmann, Heike, Susan VanderPlas, and Yawei Ge. 2022. *ggpcp: Parallel Coordinate Plots in the 'ggplot2' Framework*. R package version 0.2.0, `https://CRAN.R-project.org/package=ggpcp`.

Huang, Gao, Zhuang Liu, Laurens Van Der Maaten, and Kilian Q Weinberger. 2017. "Densely connected convolutional networks." In *Proceedings of the IEEE conference on computer vision and pattern recognition*, 4700–4708.

Jeppson, Haley, Heike Hofmann, and Di Cook. 2021. *ggmosaic: Mosaic plots in the 'ggplot2' framework*. R package version 0.3.3, `https://CRAN.R-project.org/package=ggmosaic`.

Kahle, David. 2013. "mpoly: Multivariate Polynomials in R." *The R Journal* 5 (1): 162–170.

Kingma, Diederik P, and Jimmy Ba. 2014. "Adam: A method for stochastic optimization." *arXiv preprint arXiv:1412.6980* .

Krishnan, Ganesh, and Heike Hofmann. 2021. "Hierarchical Decision Ensembles-An inferential framework for uncertain Human-AI collaboration in forensic examinations." *arXiv preprint arXiv:2111.01131* .

Kullback, Solomon, and Richard A Leibler. 1951. "On information and sufficiency." *The Annals of Mathematical Statistics* 22 (1): 79–86.

Langsrud, Øyvind. 2005. "Rotation tests." *Statistics and computing* 15: 53–60.

Lee, Howard, and Yi-Ping Phoebe Chen. 2015. "Image based computer aided diagnosis system for cancer detection." *Expert Systems with Applications* 42 (12): 5356–5365.

Li, Weihao, Dianne Cook, Emi Tanaka, and Susan VanderPlas. 2024. "A plot is worth a thousand tests: Assessing residual diagnostics with the lineup protocol." *Journal of Computational and Graphical Statistics* 1–19.

Loy, Adam, and Heike Hofmann. 2013. "Diagnostic tools for hierarchical linear models." *Wiley Interdisciplinary Reviews: Computational Statistics* 5 (1): 48–61.

Loy, Adam, and Heike Hofmann. 2014. "HLMdiag: A suite of diagnostics for hierarchical linear models in R." *Journal of Statistical Software* 56: 1–28.

Loy, Adam, and Heike Hofmann. 2015. "Are you normal? The problem of confounded residual structures in hierarchical linear models." *Journal of Computational and Graphical Statistics* 24 (4): 1191–1209.

Mason, Harriet, Stuart Lee, Ursula Laa, and Dianne Cook. 2022. *cassowaryr: Compute Scagnostics on Pairs of Numeric Variables in a Data Set*. R package version 2.0.0, `https://CRAN.R-project.org/package=cassowary`.

Müller, Kirill. 2020. *here: A simpler way to find your files*. R package version 1.0.1, `https://CRAN.R-project.org/package=here`.

Nikolenko, Sergey I, et al. 2021. *Synthetic data for deep learning*. Vol. 174. Springer.

Ojeda, Sun Arthur A, Geoffrey A Solano, and Elmer C Peramo. 2020. "Multivariate time series imaging for short-term precipitation forecasting using convolutional neural networks." In *2020 International Conference on Artificial Intelligence in Information and Communication (ICAIIC)*, 33–38. IEEE.

O'Malley, Tom, Elie Bursztein, James Long, François Chollet, Haifeng Jin, Luca Invernizzi, et al. 2019. "Keras Tuner." `https://github.com/keras-team/keras-tuner`.

Pedersen, Thomas Lin. 2022. *patchwork: The composer of plots*. R package version 1.1.2, `https://CRAN.R-project.org/package=patchwork`.

Ramsey, James Bernard. 1969. "Tests for specification errors in classical linear least-squares regression analysis." *Journal of the Royal Statistical Society: Series B (Methodological)* 31

(2): 350–371.

Rawat, Waseem, and Zenghui Wang. 2017. "Deep convolutional neural networks for image classification: A comprehensive review." *Neural computation* 29 (9): 2352–2449.

Series, BT. 2011. "Studio encoding parameters of digital television for standard 4: 3 and wide-screen 16: 9 aspect ratios." *International Telecommunication Union, Radiocommunication Sector* .

Shapiro, Samuel Sanford, and Martin B Wilk. 1965. "An analysis of variance test for normality (complete samples)." *Biometrika* 52 (3/4): 591–611.

Silverman, Bernard W. 2018. *Density estimation for statistics and data analysis*. Routledge.

Simonyan, Karen, and Andrew Zisserman. 2014. "Very deep convolutional networks for large-scale image recognition." *arXiv preprint arXiv:1409.1556* .

Singh, Karamjit, Garima Gupta, Lovekesh Vig, Gautam Shroff, and Puneet Agarwal. 2017. "Deep convolutional neural networks for pairwise causality." *arXiv preprint arXiv:1701.00597* .

Tukey, John W, and Paul A Tukey. 1985. "Computer graphics and exploratory data analysis: An introduction." In *Proceedings of the sixth annual conference and exposition: computer graphics*, Vol. 85, 773–785.

Ushey, Kevin, JJ Allaire, and Yuan Tang. 2024. *reticulate: Interface to 'Python'*. R package version 1.35.0, `https://CRAN.R-project.org/package=reticulate`.

Wang, Zhou, Alan C Bovik, Hamid R Sheikh, and Eero P Simoncelli. 2004. "Image quality assessment: from error visibility to structural similarity." *IEEE transactions on image processing* 13 (4): 600–612.

Wickham, Hadley, Mara Averick, Jennifer Bryan, Winston Chang, Lucy D'Agostino McGowan, Romain François, Garrett Grolemund, et al. 2019. "Welcome to the tidyverse." *Journal of Open Source Software* 4 (43): 1686.

Widen, Holly M, James B Elsner, Stephanie Pau, and Christopher K Uejio. 2016. "Graphical inference in geographical research." *Geographical Analysis* 48 (2): 115–131.

Wilkinson, Leland, Anushka Anand, and Robert Grossman. 2005. "Graph-theoretic scagnostics." In *Information Visualization, IEEE Symposium on*, 21–21. IEEE Computer Society.

Xie, Yihui. 2014. "knitr: A comprehensive tool for reproducible research in R." In *Implementing reproducible computational research*, edited by Victoria Stodden, Friedrich Leisch, and Roger D. Peng. Chapman and Hall/CRC. ISBN 978-1466561595, `http://www.crcpress.com/product/isbn/9781466561595`.

Xie, Yihui, Christophe Dervieux, and Emily Riederer. 2020. *R Markdown cookbook*. Boca Raton, Florida: Chapman and Hall/CRC. ISBN 9780367563837, https://bookdown.org/yihui/rmarkdown-cookbook.

Zeileis, Achim, and Torsten Hothorn. 2002. “Diagnostic checking in regression relationships.” *R News* 2 (3): 7–10.

Zhang, Ye, Yi Hou, Shilin Zhou, and Kewei Ouyang. 2020. “Encoding time series as multi-scale signed recurrence plots for classification using fully convolutional networks.” *Sensors* 20 (14): 3818.

Zhu, Hao. 2021. *kableExtra: Construct complex table with kable and pipe syntax*. R package version 1.3.4, https://CRAN.R-project.org/package=kableExtra.